

NIRBHAY GUPTA

90119976

MODULE 2 – SQL FUNDAMENTALS (DDL & DML)

Q1.

1. A customer's PAN number, always exactly 10 characters.

Data Type: CHAR(10)

Reason: PAN always contains exactly 10 characters, so CHAR(10) is appropriate for fixed-length data.

2. The interest amount credited on a fixed deposit at maturity.

Data Type: NUMERIC(15,2)

Reason: NUMERIC provides exact decimal precision, which is important for storing monetary amounts accurately.

3. The number of days a deposit runs, maximum 3,650.

Data Type: SMALLINT

Reason: The value is a whole number and the maximum of 3,650 is easily within the range of SMALLINT.

4. The exact moment a UPI payment was authorised.

Data Type: TIMESTAMP

Reason: TIMESTAMP stores both the date and exact time of an event.

5. The date a loan instalment falls due.

Data Type: DATE

Reason: Only the calendar date is required, not the time of day.

6. A free-text branch note about a customer complaint.

Data Type: TEXT

Reason: TEXT allows variable-length free-form notes without imposing an unnecessary fixed limit.

7. A running counter of every transaction the bank has ever posted.

Data Type: BIGINT

Reason: A transaction counter can become very large over the lifetime of a bank, so BIGINT provides sufficient range.

8. Whether a deposit auto-renews.

Data Type: BOOLEAN

Reason: The value represents a simple true/false condition.

Q2.

SQL demonstration

```
SELECT 0.1::FLOAT8 + 0.2::FLOAT8 AS float_result;
```

```
SELECT 0.1::NUMERIC + 0.2::NUMERIC AS numeric_result;
```

Business Impact

FLOAT can introduce rounding errors because it uses approximate floating-point representation, which is unsuitable for financial amounts where exact monetary values are required.

Q3.

Constraint	What it guarantees	One ICICI Bank example
NOT NULL	A column must always contain a value.	customer.first_name must be provided.

Constraint	What it guarantees	One ICICI Bank example
UNIQUE	No two rows can contain the same value in the constrained column.	fixed_deposit.receipt_number must be unique.
PRIMARY KEY	Uniquely identifies each row and cannot be NULL.	customer.customer_id uniquely identifies a customer.
FOREIGN KEY	A value must correspond to an existing value in the referenced table.	account.customer_id must refer to an existing customer.
CHECK	Ensures that column values satisfy a specified condition.	account.balance >= 0.
DEFAULT	Automatically supplies a value when one is not provided.	fixed_deposit.status can default to ACTIVE.

Q4.

A **PRIMARY KEY** uniquely identifies each row and cannot contain NULL, whereas a **UNIQUE** constraint prevents duplicate values but can allow NULL depending on the database and constraint definition.

A table can have **only one PRIMARY KEY**, whereas it can have **multiple UNIQUE constraints**. deposit_id was chosen as the primary key because it is a stable internal identifier for each fixed deposit. receipt_number is better treated as a unique business identifier because it identifies the deposit from a business perspective but may have formatting or business-generation requirements that are separate from the table's internal row identity.

Q5.

Dimension	DELETE	TRUNCATE	DROP
What is removed	Selected rows or all rows from a table	All rows from the table	The entire table, including its structure
WHERE allowed?	Yes	No	No
Can it be rolled back?	Yes, within a transaction in PostgreSQL	Yes, within a transaction in PostgreSQL	Yes, within a transaction in PostgreSQL
Do triggers fire?	DELETE triggers fire	Normal row-level DELETE triggers do not fire	Table-level removal occurs; DELETE triggers do not fire
Speed on a large table	Generally slower because individual rows are processed	Generally faster for removing all rows	Fast for removing the entire table, but destroys the table structure

Example

```
DELETE FROM fixed_deposit
WHERE branch_id = 1;
removes selected rows.

TRUNCATE TABLE fixed_deposit;
```

removes all rows but keeps the table.

```
DROP TABLE fixed_deposit;
```

removes the table itself.

Q6.

The query is incorrect because it deletes transactions older than **90 days**, whereas the bank is required to retain transaction history for **seven years**.

Deleting old transactions permanently would violate the required retention period and could also remove information needed for regulatory reporting, auditing and investigation.

The bank should retain at least seven years of transaction history according to the applicable retention requirement and use a controlled archival strategy for older data if required. Archived data should remain accessible for regulatory and audit purposes.

Correct retention condition

If data older than seven years is legally permitted to be removed, the condition would be based on seven years rather than 90 days:

```
DELETE FROM account_txn
```

```
WHERE txn_date < CURRENT_DATE - INTERVAL '7 years';
```

However, this should only be executed after confirming the bank's applicable retention policy and regulatory requirements.

Q7.

1.

```
SELECT NULL = NULL;
```

Output:

NULL

Reason: NULL represents an unknown value, so SQL cannot determine that one unknown value equals another unknown value.

2.

```
SELECT NULL <> NULL;
```

Output:

NULL

Reason: The comparison with NULL produces an unknown result rather than TRUE or FALSE.

3.

```
SELECT 5000 + NULL;
```

Output:

NULL

Reason: Arithmetic involving NULL produces NULL.

4.

```
SELECT COALESCE(NULL, NULL, 'FALLBACK');
```

Output:

FALLBACK

Reason: COALESCE returns the first non-NULL value.

5.

```
SELECT 'A' || NULL || 'B';
```

Output:

NULL

Reason: In PostgreSQL, concatenation with NULL results in NULL.

The five queries are exactly the set specified in the exercise.

Q8.

The two queries are:

Query 1

```
SELECT COUNT(*)
FROM fixed_deposit
WHERE branch_id = 1
   OR branch_id = 2
   AND principal_amount > 500000;
```

Query 2

```
SELECT COUNT(*)
FROM fixed_deposit
WHERE (branch_id = 1 OR branch_id = 2)
   AND principal_amount > 500000;
```

The two queries return different row counts because **AND has higher operator precedence than OR**.

Therefore, Query 1 is interpreted as:

```
WHERE branch_id = 1
   OR (branch_id = 2 AND principal_amount > 500000);
```

This means **all deposits from branch 1** are included regardless of their principal amount, while the ₹500,000 condition applies only to branch 2.

Query 2 explicitly uses parentheses:

```
WHERE (branch_id = 1 OR branch_id = 2)
   AND principal_amount > 500000;
```

Here, the ₹500,000 condition applies to **both branches**.

Q9.

The defect occurs when `txn_date` is a **timestamp** rather than a date. The upper boundary:

```
DATE '2026-03-31'
```

represents midnight at the beginning of March 31. Therefore, transactions occurring later on March 31 can be excluded.

For example:

```
2026-03-31 10:30:00
```

is greater than:

```
2026-03-31 00:00:00
```

Correct predicate

```
WHERE txn_date >= DATE '2026-01-01'
   AND txn_date < DATE '2026-04-01'
```

This includes the entire period from January 1 through March 31, including transactions occurring at any time on March 31.

Q10.

SQL clauses are logically evaluated in an order where the **WHERE clause is processed before the SELECT list**, while **ORDER BY is processed after the SELECT list**.

Therefore, a column alias created in the SELECT list does not yet exist when the WHERE clause is evaluated, so it cannot normally be referenced there.

However, by the time ORDER BY is evaluated, the SELECT list and its aliases are already available.

Example

This works:

```
SELECT principal_amount * rate / 100 AS interest
FROM fixed_deposit
ORDER BY interest DESC;
```

But this does not normally work:

```
SELECT principal_amount * rate / 100 AS interest
FROM fixed_deposit
WHERE interest > 5000;
```

Q11.

```
CREATE TABLE recurring_deposit
(
    rd_id INTEGER
        CONSTRAINT pk_recurring_deposit PRIMARY KEY,

    customer_id INTEGER
        CONSTRAINT fk_rd_customer
        REFERENCES customer(customer_id),

    branch_id INTEGER
        CONSTRAINT fk_rd_branch
        REFERENCES branch(branch_id),

    rd_number VARCHAR(20)
        CONSTRAINT uq_rd_number UNIQUE,

    monthly_amount NUMERIC(15,2)
        CONSTRAINT ck_rd_monthly_amount CHECK (monthly_amount >= 500),

    interest_rate NUMERIC(5,2)
        CONSTRAINT ck_rd_interest_rate
        CHECK (interest_rate > 0 AND interest_rate <= 12),

    tenure_months INTEGER
        CONSTRAINT ck_rd_tenure
        CHECK (tenure_months BETWEEN 6 AND 120),

    start_date DATE
        CONSTRAINT nn_rd_start_date NOT NULL
        DEFAULT CURRENT_DATE,
```

```

maturity_date DATE
    CONSTRAINT nn_rd_maturity_date NOT NULL,

installments_paid INTEGER
    CONSTRAINT ck_rd_installments_paid
    CHECK (installments_paid >= 0)
    DEFAULT 0,

status VARCHAR(12)
    CONSTRAINT ck_rd_status
    CHECK (status IN ('ACTIVE','MATURED','DEFAULTED','CLOSED'))
    DEFAULT 'ACTIVE',

CONSTRAINT ck_rd_maturity_date
    CHECK (maturity_date > start_date)
);

```

Constraints used

Constraint	Purpose
pk_recurring_deposit	Primary key for rd_id
fk_rd_customer	Customer must exist
fk_rd_branch	Branch must exist
uq_rd_number	RD number must be unique
ck_rd_monthly_amount	Minimum ₹500
ck_rd_interest_rate	Rate > 0 and ≤ 12
ck_rd_tenure	Tenure between 6 and 120 months
nn_rd_start_date	Start date mandatory
nn_rd_maturity_date	Maturity date mandatory
ck_rd_maturity_date	Maturity must be after start date
ck_rd_installments_paid	Cannot be negative
ck_rd_status	Only four valid statuses

Q12.

1. RD with monthly amount of 100

```

INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date)
VALUES
(9001, 1001, 1, 'RDINVALID001', 100,
7.10, 12, DATE '2026-03-01', DATE '2027-03-01');

```

Expected: Rejected by ck_rd_monthly_amount.

2. RD for non-existing customer 5555

```

INSERT INTO recurring_deposit

```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date)
VALUES
(9002, 5555, 1, 'RDINVALID002', 5000,
7.10, 12, DATE '2026-03-01', DATE '2027-03-01');
```

Expected: Rejected by fk_rd_customer.

3. Maturity date before start date

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date)
VALUES
(9003, 1001, 1, 'RDINVALID003', 5000,
7.10, 12, DATE '2026-03-10', DATE '2026-03-01');
```

Expected: Rejected by ck_rd_maturity_date.

4. Two RDs with the same RD number

First insert:

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date)
VALUES
(9004, 1001, 1, 'RD_DUPLICATE',
5000, 7.10, 12, DATE '2026-03-01', DATE '2027-03-01');
```

Then insert another with the same rd_number:

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date)
VALUES
(9005, 1005, 5, 'RD_DUPLICATE',
6000, 7.20, 12, DATE '2026-03-01', DATE '2027-03-01');
```

Expected: The second statement is rejected by uq_rd_number.

5. RD with status PAUSED

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date, status)
VALUES
(9006, 1001, 1, 'RDINVALID006',
5000, 7.10, 12, DATE '2026-03-01',
DATE '2027-03-01', 'PAUSED');
```

Expected: Rejected by ck_rd_status.

6. RD with installments_paid = -3

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, start_date, maturity_date,
```

```
installments_paid)
VALUES
(9007, 1001, 1, 'RDINVALID007',
 5000, 7.10, 12, DATE '2026-03-01',
 DATE '2027-03-01', -3);
```

Expected: Rejected by ck_rd_installments_paid.

For your Word document: paste the **actual error displayed by pgAdmin** below each query.

Q13.

Insert the three recurring deposits in a single statement.

The exercise gives the three rows on page 5.

SQL Answer

```
INSERT INTO recurring_deposit
(
  rd_id,
  customer_id,
  branch_id,
  rd_number,
  monthly_amount,
  interest_rate,
  tenure_months,
  start_date,
  maturity_date
)
VALUES
(
  8001, 1001, 1, 'RD2026000001',
  5000, 7.10, 24,
  DATE '2026-03-01',
  DATE '2028-03-01'
),
(
  8002, 1005, 5, 'RD2026000002',
  2500, 6.85, 12,
  DATE '2026-03-05',
  DATE '2027-03-05'
),
(
  8003, 1012, 4, 'RD2026000003',
  10000, 7.35, 60,
  DATE '2026-03-10',
  DATE '2031-03-10'
);
```

Verify

```
SELECT *
FROM recurring_deposit
WHERE rd_id IN (8001, 8002, 8003)
```

ORDER BY rd_id;

Q14.

Step 1 — Add column

```
ALTER TABLE recurring_deposit
```

```
ADD COLUMN nominee_name VARCHAR(60);
```

Because the column is nullable, existing rows are allowed to have NULL.

Step 2 — Set nominee

```
UPDATE recurring_deposit
```

```
SET nominee_name = 'Meera Reddy'
```

```
WHERE rd_id = 8001;
```

Step 3 — Verify

```
SELECT rd_id, rd_number, nominee_name
```

```
FROM recurring_deposit
```

```
WHERE rd_id = 8001;
```

Q15.

Step 1 — Check existing violations

```
SELECT *
```

```
FROM recurring_deposit
```

```
WHERE monthly_amount < 1000;
```

Expected result

If the three inserted rows are present, this should return **no rows**, because their monthly amounts are ₹5,000, ₹2,500 and ₹10,000.

Step 2 — Drop old constraint

```
ALTER TABLE recurring_deposit
```

```
DROP CONSTRAINT ck_rd_monthly_amount;
```

Step 3 — Add new constraint

```
ALTER TABLE recurring_deposit
```

```
ADD CONSTRAINT ck_rd_monthly_amount
```

```
CHECK (monthly_amount >= 1000);
```

Verify constraint

```
SELECT conname, pg_get_constraintdef(oid)
```

```
FROM pg_constraint
```

```
WHERE conname = 'ck_rd_monthly_amount';
```

Conclusion

The existing minimum of ₹500 was replaced with a new minimum monthly instalment of ₹1,000.

Q16.

Step 1 — Try directly

```
ALTER TABLE recurring_deposit
```

```
ADD COLUMN scheme_code VARCHAR(8) NOT NULL;
```

Expected result

This should fail because the table already contains rows and PostgreSQL cannot assign a non-NULL value to the new column for those existing rows.

Step 2 — Add the column without NOT NULL

```
ALTER TABLE recurring_deposit  
ADD COLUMN scheme_code VARCHAR(8);
```

Step 3 — Populate existing rows

```
UPDATE recurring_deposit  
SET scheme_code = 'RD2026'  
WHERE scheme_code IS NULL;
```

Step 4 — Make the column mandatory

```
ALTER TABLE recurring_deposit  
ALTER COLUMN scheme_code SET NOT NULL;
```

Verify

```
SELECT rd_id, scheme_code  
FROM recurring_deposit  
ORDER BY rd_id;
```

Conclusion

A mandatory column must first be populated for existing rows before the NOT NULL constraint can be applied.

Q17.

SQL

```
SELECT receipt_number,  
       principal_amount,  
       rate,  
       maturity_date  
FROM fixed_deposit  
WHERE principal_amount >= 500000  
ORDER BY principal_amount DESC;
```

Q18.

SQL

```
SELECT receipt_number,  
       principal_amount,  
       rate,  
       booked_date,  
       maturity_date  
FROM fixed_deposit  
WHERE booked_date >= DATE '2026-03-01'  
      AND booked_date < DATE '2026-03-11'  
ORDER BY booked_date,  
       principal_amount DESC;
```

Why < 11 March?

This approach includes the **entire day of 10 March**, even if booked_date is a timestamp containing a time component.

Q19.

Query 1 — No nominee

```
SELECT receipt_number,  
       principal_amount,  
       nominee_name  
FROM fixed_deposit  
WHERE nominee_name IS NULL;
```

Query 2 — Nominee is not Priya Verma, including NULL

```
SELECT receipt_number,  
       principal_amount,  
       nominee_name  
FROM fixed_deposit  
WHERE nominee_name <> 'Priya Verma'  
   OR nominee_name IS NULL;
```

Q20.**SQL**

```
SELECT receipt_number,  
       principal_amount,  
       ROUND((principal_amount * rate / 100), 2) AS annual_interest  
FROM fixed_deposit  
ORDER BY annual_interest DESC  
LIMIT 5;
```

Q21.

```
SELECT fd.receipt_number,  
       c.first_name || ' ' || c.last_name AS customer_full_name,  
       fd.principal_amount,  
       CASE  
         WHEN fd.principal_amount >= 1000000  
           THEN 'HIGH VALUE'  
         WHEN fd.principal_amount >= 100000  
           THEN 'STANDARD'  
         ELSE 'SMALL'  
       END AS segment  
FROM fixed_deposit fd  
JOIN customer c  
  ON fd.customer_id = c.customer_id  
ORDER BY  
  CASE  
    WHEN fd.principal_amount >= 1000000 THEN 1  
    WHEN fd.principal_amount >= 100000 THEN 2  
    ELSE 3  
  END,  
  fd.principal_amount DESC;
```

Q22.

```

SELECT txn_id,
       account_id,
       amount,
       narration
FROM account_txn
WHERE narration ILIKE '%UPI%'
      AND amount > 50000
ORDER BY amount DESC
LIMIT 10;

```

SQL Server

```

SELECT TOP 10
       txn_id,
       account_id,
       amount,
       narration
FROM account_txn
WHERE narration LIKE '%UPI%'
      AND amount > 50000
ORDER BY amount DESC;

```

Oracle

```

SELECT txn_id,
       account_id,
       amount,
       narration
FROM account_txn
WHERE UPPER(narration) LIKE '%UPI%'
      AND amount > 50000
ORDER BY amount DESC
FETCH FIRST 10 ROWS ONLY;

```

Key difference

Database	Method
PostgreSQL	LIMIT 10
SQL Server	TOP 10
Oracle	FETCH FIRST 10 ROWS ONLY

Q23.

Step 1 — Identify the deposit

```

SELECT *
FROM fixed_deposit
WHERE deposit_id = 7009;

```

Step 2 — Start transaction and update

```

BEGIN;

UPDATE fixed_deposit
SET status = 'BROKEN',
    auto_renew = 'N'

```

```
WHERE deposit_id = 7009;
```

Step 3 — Verify

```
SELECT deposit_id,  
       receipt_number,  
       status,  
       auto_renew  
FROM fixed_deposit  
WHERE deposit_id = 7009;
```

Expected changed values

```
status    = BROKEN  
auto_renew = N
```

Step 4 — Rollback

```
ROLLBACK;
```

Step 5 — Confirm original state

```
SELECT deposit_id,  
       receipt_number,  
       status,  
       auto_renew  
FROM fixed_deposit  
WHERE deposit_id = 7009;
```

Q24.

Step 1 — Find the deposits first

```
SELECT *  
FROM fixed_deposit  
WHERE status = 'ACTIVE'  
      AND maturity_date <= DATE '2026-06-30';
```

Record the number of rows returned.

Step 2 — Start transaction

```
BEGIN;
```

Step 3 — Update

```
UPDATE fixed_deposit  
SET status = 'MATURED'  
WHERE status = 'ACTIVE'  
      AND maturity_date <= DATE '2026-06-30';
```

pgAdmin will show something like:

```
UPDATE <number>
```

The <number> should match the number of rows returned by the SELECT.

Step 4 — Confirm

```
SELECT COUNT(*) AS matured_count  
FROM fixed_deposit  
WHERE status = 'MATURED'  
      AND maturity_date <= DATE '2026-06-30';
```

Important

The count should correspond to the rows affected by the update, subject to any rows that were already MATURED.

Step 5 — Rollback

ROLLBACK;

Conclusion

Q25.

Step 1 — Check current values

```
SELECT rd_id,  
       status,  
       installments_paid  
FROM recurring_deposit  
WHERE rd_id = 8002;
```

Step 2 — Mark it as DEFAULTED

```
UPDATE recurring_deposit  
SET status = 'DEFAULTED'  
WHERE rd_id = 8002;  
Notice that installments_paid is not changed.
```

Step 3 — Verify

```
SELECT rd_id,  
       status,  
       installments_paid  
FROM recurring_deposit  
WHERE rd_id = 8002;
```

Step 4 — Close the RD instead of deleting it

```
UPDATE recurring_deposit  
SET status = 'CLOSED'  
WHERE rd_id = 8002;
```

Step 5 — Final verification

```
SELECT rd_id,  
       status,  
       installments_paid  
FROM recurring_deposit  
WHERE rd_id = 8002;
```

Q26.

I would not store principal_amount as VARCHAR(30) in the banking database because it is a monetary value and should be stored using an exact numeric data type.

I would insist that the vendor provide a **standard numeric representation**, without commas, currency symbols or text such as INR.

The value should be validated and cleaned in the **data ingestion/staging layer** before it reaches the core database.

After validation, the cleaned value should be stored as an exact type such as NUMERIC(15,2).

For example:

"250000" → 250000.00

"2,50,000.00" → 250000.00

"250000 INR" → 250000.00

The cleaning process should also reject malformed or ambiguous values rather than silently converting them.

Q27.

I would choose **RESTRICT** for `fixed_deposit.customer_id` because a customer's deposits should not be automatically deleted when the customer record is closed. When a customer leaves the bank, the bank normally retains financial records for audit, legal and regulatory requirements rather than physically deleting them. `ON DELETE CASCADE` could unintentionally remove fixed-deposit records when a customer record is deleted. **RESTRICT** forces the bank to handle the customer's deposits explicitly before the customer record can be removed. This provides greater protection against accidental deletion of important financial history.

Q28.

Column

`authorised_by_employee_id` INTEGER

Nullability

The column should allow NULL because the requirement says "**where applicable**". Some transactions may not require an employee authorisation.

Foreign key

Yes. It should reference the employee table:

FOREIGN KEY (`authorised_by_employee_id`)

REFERENCES `employee(employee_id)`

SQL

```
ALTER TABLE account_txn
```

```
ADD COLUMN authorised_by_employee_id INTEGER;
```

Then:

```
ALTER TABLE account_txn
```

```
ADD CONSTRAINT fk_account_txn_authorised_employee
```

```
FOREIGN KEY (authorised_by_employee_id)
```

```
REFERENCES employee(employee_id);
```

What about the 40 million existing rows?

Existing rows should initially have NULL in the new column because historical transactions may not have this information.

I would **not invent employee IDs** for those records. The bank should determine whether historical authorisation information exists in another trusted source and populate it through a controlled migration where possible.

Q29.

Approach 1 — CHECK constraint

Advantage 1

It is **simple and easy to understand** because the allowed statuses are directly defined on the `fixed_deposit` table.

Advantage 2

It avoids an additional lookup table and join, making the design **simpler for a small, stable set of status values**.

Approach 2 — Separate lookup table + Foreign Key

Advantage 1

The allowed statuses are maintained in a **central table**, making it easier to add or manage statuses without modifying the table constraint.

Advantage 2

The lookup table can contain additional information about each status, such as a description, display name or whether the status is active.

My choice

I would choose the **CHECK constraint** for `fixed_deposit.status` because the table has a small and stable set of four known status values. A separate lookup table would add complexity without much benefit unless the status list becomes larger, changes frequently, or requires additional metadata.

Q30.

First, I would immediately stop further changes to the affected account table if possible.

I would determine how many rows were affected and record the exact time and statement.

Since autocommit is enabled, I would check whether a reliable backup or audit history is available.

I would identify the correct previous status for each affected account from a trusted source.

I would restore the correct values using a controlled and reviewed recovery operation.

I would verify the restored data using counts and targeted checks.

I would document the incident and preserve the relevant logs for audit purposes.

The team should require a review or approval for production-wide UPDATE and DELETE statements.

The team should also use transactions and safer execution practices before applying production changes.

These controls reduce the chance of an accidental broad update while focusing on improving the process rather than blaming an individual.